Методические материалы к лабораторной работе

«Решение многоэтапных оптимизационных задач методом динамического программирования»

1. Цель и содержание работы

Цель работы — освоить полный цикл решения многоэтапной оптимизационной задачи: от математической постановки и вывода рекуррентного соотношения Беллмана до ручных вычислений, программной реализации и проверки результата.

В ходе работы необходимо:

1. Составить математическую модель задачи в соответствии с индивидуальным вариантом.
2. Представить задачу как последовательность этапов принятия решений.
3. Определить состояния системы, допустимые управления и переходы между состояниями.
4. Вывести и записать рекуррентное соотношение Беллмана.
5. Выполнить ручное решение задачи с заполнением расчетных таблиц.
6. Восстановить оптимальную последовательность решений.
7. Разработать программу, реализующую метод динамического программирования.
8. Сопоставить результаты ручного и программного решения.

Основное требование: необходимо получить не только оптимальное значение целевой функции, но и **стратегию** — последовательность допустимых решений, обеспечивающую это значение.

Ниже приведены общий порядок выполнения работы и полностью разобранный учебный пример. Поскольку конкретные варианты заданий не представлены, пример построен на задаче распределения ограниченного ресурса.

2. Теоретические сведения

2.1. Многоэтапная оптимизационная задача

В динамическом программировании процесс принятия решений представляют как последовательность этапов. На каждом этапе система находится в некотором состоянии; выбранное управление определяет непосредственный результат и состояние следующего этапа. Этапы могут соответствовать моментам времени, но могут вводиться и искусственно — например, по номеру проекта, которому выделяется ресурс.[1] [2]

Рассмотрим детерминированную задачу с конечным числом этапов:

$$k = 1, 2, \dots, N.$$

Обозначим:

- s_k — состояние системы перед этапом k;
- u_k — управление на этапе k;
- $U_k(s_k)$ — множество допустимых управлений;
- $f_k(s_k, u_k)$ — функция перехода;
- $g_k(s_k, u_k)$ — результат этапа;
- $h(s_{N+1})$ — терминальный результат.

Математическая постановка задачи максимизации:

$$\max_{u_1, \dots, u_N} \left[\sum_{k=1}^{N} g_k(s_k, u_k) + h(s_{N+1}) \right]$$

при условиях

$$s_{k+1} = f_k(s_k, u_k), \qquad u_k \in U_k(s_k), \qquad s_1 = s^{(0)}.$$

Такая постановка объединяет аддитивную целевую функцию, динамику состояния и ограничения на решения; она соответствует стандартной схеме динамического программирования для конечного горизонта.[3] [4]

2.2. Принцип оптимальности Беллмана

Принцип оптимальности: оставшаяся часть оптимальной стратегии должна быть оптимальной для подзадачи, начинающейся из состояния, достигнутого после уже принятых решений. [2]

Для применения этого принципа состояние необходимо выбирать так, чтобы оно содержало всю информацию о прошлом, существенную для дальнейших решений. Если выбранного описания недостаточно, состояние следует дополнить.[4] [5]

Например, при распределении ресурса между независимыми проектами состоянием может быть остаток ресурса. Если же доходность проекта зависит от того, какие проекты уже профинансированы, одного остатка недостаточно.

2.3. Функция Беллмана

Определим

$$V_k(s)$$

как максимальный суммарный результат, который можно получить на этапах от k до N, если перед этапом k система находится в состоянии s.

Тогда

$$V_k(s) = \max_{u \in U_k(s)} \left[g_k(s,u) + V_{k+1}\big(f_k(s,u)\big) \right],$$

а граничное условие имеет вид

$$V_{N+1}(s) = h(s).$$

Функции V_k вычисляют в обратном порядке — от последнего этапа к первому. Для восстановления оптимальной стратегии дополнительно сохраняют управления, на которых достигается максимум. [2] [3]

Для задачи минимизации используется аналогичная запись с оператором $\min$.

3. Задание на лабораторную работу

Для своего варианта задания выполните следующие действия.

3.1. Математическая постановка

Укажите:

1. Исходные данные и их единицы измерения.
2. Переменные решения.
3. Целевую функцию.
4. Ограничения.
5. Начальное состояние.
6. Условия завершения процесса.
7. Смысл требуемого результата.

Сначала запишите исходную задачу оптимизации, затем представьте ее в многоэтапной форме.

3.2. Построение многоэтапной модели

Заполните таблицу.

Элемент модели	Что необходимо указать
Этап k	Какой объект или момент процесса ему соответствует

Элемент модели	Что необходимо указать
Состояние s_k	Какая информация необходима для дальнейших решений
Управление u_k	Какое решение принимается на этапе
Допустимые управления $U_k(s_k)$	Какие решения разрешены в данном состоянии
Переход $f_k(s_k, u_k)$	Как изменяется состояние
Результат $g_k(s_k, u_k)$	Какой вклад этап вносит в целевую функцию
Начальное условие	Из какого состояния начинается процесс
Терминальное условие	Какие конечные состояния допустимы

3.3. Вывод соотношения Беллмана

Недостаточно привести общую формулу. Необходимо:

1. Определить функцию Беллмана словами.
2. Рассмотреть произвольное допустимое решение текущего этапа.
3. Записать непосредственный результат этого решения.
4. Указать состояние следующего этапа.
5. Добавить оптимальный результат оставшихся этапов.
6. Выполнить максимизацию или минимизацию по допустимым решениям.
7. Записать граничные условия.

3.4. Ручное решение

При ручном решении следует:

- вычислять значения функции Беллмана от последнего этапа к первому;
- для каждого рассматриваемого состояния перечислять допустимые управления;
- показывать вычисляемые значения целевой функции;
- сохранять оптимальные управления;
- после заполнения таблиц выполнить прямой проход из начального состояния;
- проверить допустимость полученного плана и значение целевой функции.

Если несколько управлений дают одинаковый оптимум, это необходимо отразить в таблице.

3.5. Программная реализация

Программа должна:

- получать исходные данные отдельно от алгоритма;
- вычислять значения функций Беллмана;

- сохранять оптимальные управления;
- восстанавливать хотя бы один оптимальный план;
- выводить значение целевой функции и найденный план;
- проверять ограничения;
- позволять сопоставить программные таблицы с ручными расчетами.

4. Учебный пример: распределение ресурса между проектами

4.1. Условие

Имеется 4 единицы ресурса, которые необходимо полностью распределить между тремя независимыми проектами.

Ресурс выделяется целыми неотрицательными порциями. Доход каждого проекта зависит только от объема выделенного ему ресурса.

Выделенный ресурс u	Доход проекта 1 $r_1(u)$	Доход проекта 2 $r_2(u)$	Доход проекта 3 $r_3(u)$
0	0	0	0
1	3	2	1
2	5	5	4
3	6	7	6
4	7	8	9

Требуется найти распределение, обеспечивающее максимальный суммарный доход.

4.2. Исходная математическая постановка

Пусть x_i — объем ресурса, выделенный проекту i.

Тогда

$$
[
\max F(x_1,x_2,x_3)
$$

r_1(x_1)+r_2(x_2)+r_3(x_3)
]

при ограничениях

$$x_1 + x_2 + x_3 = 4,$$

$$x_i \in \{0, 1, 2, 3, 4\}, \qquad i = 1, 2, 3.$$

Здесь требуется использовать весь ресурс. Это условие должно учитываться не только при проверке итогового плана, но и в граничных условиях динамического

программирования.

4.3. Многоэтапная постановка

Введем следующие обозначения:

- этап k — распределение ресурса проекту k;
- состояние s — остаток ресурса перед данным этапом;
- управление u — ресурс, выделяемый текущему проекту;
- результат этапа — $r_k(u)$;
- следующее состояние — $s - u$.

Начальное состояние:

$$s_1 = 4.$$

Допустимые управления:

$$U_k(s) = \{0, 1, \dots, s\}.$$

Уравнение перехода:

$$s_{k+1} = s_k - u_k.$$

Терминальное требование:

$$s_4 = 0.$$

Порядок рассмотрения проектов здесь задает вычислительные этапы, а не последовательность инвестирования во времени.

5. Вывод соотношения Беллмана для примера

Определим $V_k(s)$ как максимальный доход от проектов с номерами $k, \dots, 3$, если между ними необходимо полностью распределить s единиц ресурса.

Пусть проекту k выделено u единиц. Тогда:

1. Доход текущего проекта равен $r_k(u)$.
2. Для остальных проектов остается $s - u$ единиц.
3. Максимальный доход от остальных проектов равен $V_{k+1}(s - u)$.

Следовательно, результат при фиксированном u:

$$r_k(u) + V_{k+1}(s - u).$$

Выбирая наилучшее допустимое управление, получаем

$$V_k(s) = \max_{u=0,\dots,s} [r_k(u) + V_{k+1}(s - u)], \qquad k = 1, 2, 3.$$

Чтобы обеспечить полное использование ресурса, зададим

$$V_4(0) = 0,$$

$$V_4(s) = -\infty, \qquad s > 0.$$

Значение $-\infty$ обозначает недопустимое завершение: проекты закончились, но ресурс остался.

Оптимальные управления сохраняются в виде множества

$$A_k(s) = \arg\max_{u=0,\dots,s} [r_k(u) + V_{k+1}(s-u)].$$

Искомое оптимальное значение:

$$F^* = V_1(4).$$

Если по условию разрешалось бы не использовать часть ресурса без штрафа, терминальное условие было бы другим: $V_4(s) = 0$ для всех допустимых остатков. Поэтому граничные условия следует выводить из содержательной постановки, а не выбирать формально.

6. Ручное решение учебного примера

6.1. Третий этап

Остался только проект 3. Весь остаток должен быть выделен ему:

$$V_3(s) = r_3(s), \qquad A_3(s) = \{s\}.$$

Остаток s	$V_3(s)$	Оптимальное управление
0	0	0
1	1	1
2	4	2
3	6	3
4	9	4

6.2. Второй этап

Используем соотношение

$$V_2(s) = \max_{u=0,\dots,s} [r_2(u) + V_3(s-u)].$$

Например, при $s = 4$:

$$V_2(4) = \max \begin{Bmatrix} 0+9, & u=0, \\ 2+6, & u=1, \\ 5+4, & u=2, \\ 7+1, & u=3, \\ 8+0, & u=4 \end{Bmatrix} = 9.$$

Максимум достигается при $u = 0$ и $u = 2$.

Полная таблица второго этапа:

s	$u=0$	$u=1$	$u=2$	$u=3$	$u=4$	$V_2(s)$	$A_2(s)$
0	0	—	—	—	—	0	$\{0\}$
1	1	2	—	—	—	2	$\{1\}$
2	4	3	5	—	—	5	$\{2\}$
3	6	6	6	7	—	7	$\{3\}$
4	9	8	9	8	8	9	$\{0,2\}$

Знак «—» означает, что управление недопустимо, поскольку превышает имеющийся остаток ресурса.

6.3. Первый этап

Используем соотношение

$$V_1(s) = \max_{u=0,\dots,s} [r_1(u) + V_2(s-u)].$$

s	$u=0$	$u=1$	$u=2$	$u=3$	$u=4$	$V_1(s)$	$A_1(s)$
0	0	—	—	—	—	0	$\{0\}$
1	2	3	—	—	—	3	$\{1\}$
2	5	5	5	—	—	5	$\{0,1,2\}$
3	7	8	7	6	—	8	$\{1\}$
4	9	10	10	8	7	10	$\{1,2\}$

Таким образом,

$$F^* = V_1(4) = 10.$$

6.4. Восстановление оптимальных решений

Начинаем из состояния $s_1 = 4$.

Первый вариант:

1. На первом этапе выбираем $u_1 = 1$.
2. Остается $s_2 = 3$.

3. По таблице второго этапа выбираем $u_2 = 3$.
4. Остается $s_3 = 0$, поэтому $u_3 = 0$.

Получаем

$$(x_1, x_2, x_3) = (1, 3, 0).$$

Проверка:

$$1 + 3 + 0 = 4,$$

$$F = 3 + 7 + 0 = 10.$$

Второй вариант:

1. На первом этапе выбираем $u_1 = 2$.
2. Остается $s_2 = 2$.
3. По таблице второго этапа выбираем $u_2 = 2$.
4. Третьему проекту выделяется $u_3 = 0$.

Получаем

$$(x_1, x_2, x_3) = (2, 2, 0),$$

$$F = 5 + 5 + 0 = 10.$$

Следовательно, в учебном примере имеются два оптимальных распределения.

7. Программная реализация на Python

Приведенная программа решает задачу полного распределения целочисленного ресурса между произвольным числом проектов. Она сохраняет все оптимальные управления в каждом состоянии, а при восстановлении одного плана выбирает первое из них.

```
from math import inf


def allocate_resource(income, resource):
    """
    income[k][u] — доход проекта k при выделении u единиц.
    resource — общий объем ресурса.

    Возвращает:
        optimum — максимальный доход;
        allocation — один оптимальный план;
        value — таблицу функций Беллмана;
        actions — таблицу всех оптимальных управлений.
    """
    if not isinstance(resource, int) or resource < 0:
        raise ValueError(
            "Объем ресурса должен быть неотрицательным целым числом."
        )
```

```
    if not income:
        raise ValueError("Необходимо задать хотя бы один проект.")

    if any(len(row) != resource + 1 for row in income):
        raise ValueError(
            "Для каждого проекта задайте доходы "
            "для всех объемов от 0 до resource."
        )

    n = len(income)

    value = [
        [-inf] * (resource + 1)
        for _ in range(n + 1)
    ]

    actions = [
        [[] for _ in range(resource + 1)]
        for _ in range(n)
    ]

    # После последнего проекта допустим только нулевой остаток.
    value[n][^1_0] = 0

    # Обратный проход.
    for k in range(n - 1, -1, -1):
        for s in range(resource + 1):
            for u in range(s + 1):
                future = value[k + 1][s - u]

                if future == -inf:
                    continue

                candidate = income[k][u] + future

                if candidate > value[k][s]:
                    value[k][s] = candidate
                    actions[k][s] = [u]

                elif candidate == value[k][s]:
                    actions[k][s].append(u)

    # Прямой проход: восстановление одного оптимального плана.
    allocation = []
    s = resource

    for k in range(n):
        u = actions[k][s][^1_0]
        allocation.append(u)
        s -= u

    actual_income = sum(
        income[k][allocation[k]]
        for k in range(n)
    )
```

```
    assert s == 0
    assert sum(allocation) == resource
    assert actual_income == value[^1_0][resource]

    return value[^1_0][resource], allocation, value, actions


income = [
    [0, 3, 5, 6, 7],
    [0, 2, 5, 7, 8],
    [0, 1, 4, 6, 9],
]

resource = 4

optimum, allocation, value, actions = allocate_resource(
    income, resource
)

print("Максимальный доход:", optimum)
print("Один оптимальный план:", allocation)

for k in range(len(income)):
    print(f"V_{k + 1}:", value[k])
    print(f"A_{k + 1}:", actions[k])
```

Ожидаемый основной результат:

```
Максимальный доход: 10
Один оптимальный план: [1, 3, 0]
```

В программе используется индексация проектов с нуля, а в математической записи — с единицы.

В данном примере все доходы целочисленные, поэтому точное сравнение значений допустимо. При адаптации программы к вычислениям с дробными числами необходимо отдельно определить правило сравнения близких результатов.

7.1. Оценка вычислительной сложности

Для каждого из N проектов программа рассматривает $B + 1$ состояний и до $B + 1$ управлений в каждом состоянии. Поэтому для данной реализации:

$$T(N, B) = O(NB^2).$$

Таблица значений занимает

$$O(NB)$$

памяти. Если хранить одно оптимальное управление для каждого состояния, таблица управлений также занимает $O(NB)$. В приведенной программе сохраняются все

равнозначные управления, поэтому в худшем случае память для них может достигать $O(NB^2)$.

Эта оценка относится к конкретной реализации задачи распределения ресурса, а не ко всем алгоритмам динамического программирования.

8. Проверка правильности программы

8.1. Сопоставление с ручным решением

Для учебного примера должны совпасть:

Проверяемый результат	Ожидаемое значение
$V_3(s), s = 0, \dots, 4$	$0,\ 1,\ 4,\ 6,\ 9$
$V_2(s), s = 0, \dots, 4$	$0,\ 2,\ 5,\ 7,\ 9$
$V_1(s), s = 0, \dots, 4$	$0,\ 3,\ 5,\ 8,\ 10$
Оптимальное значение	10
Оптимальные планы	$(1, 3, 0), (2, 2, 0)$

Совпадения только итогового значения недостаточно: необходимо проверить также восстановленный план и промежуточные таблицы.

8.2. Проверка полным перебором

Для небольшого учебного примера используйте независимую проверку полным перебором:

```
from itertools import product


def brute_force(income, resource):
    n = len(income)
    best_value = float("-inf")
    best_plans = []

    for plan in product(range(resource + 1), repeat=n):
        if sum(plan) != resource:
            continue

        result = sum(
            income[k][plan[k]]
            for k in range(n)
        )

        if result > best_value:
            best_value = result
            best_plans = [plan]

        elif result == best_value:
```

```
            best_plans.append(plan)

    return best_value, best_plans


check_value, check_plans = brute_force(income, resource)

assert optimum == check_value
assert tuple(allocation) in check_plans

print("Проверка перебором:", check_value)
print("Все оптимальные планы:", check_plans)
```

Для больших исходных данных полный перебор не требуется. Его назначение в работе — проверить корректность реализации на малом контрольном примере.

8.3. Дополнительные тесты

Проверьте программу как минимум на следующих случаях:

1. Нулевой объем ресурса.
2. Один проект.
3. Нулевые доходы всех проектов.
4. Наличие нескольких оптимальных планов.
5. Случай, когда оптимально выделить весь ресурс одному проекту.
6. Некорректные входные данные: отрицательный ресурс или неполная таблица доходов.

Для каждого теста укажите входные данные, ожидаемый результат и фактический результат.

9. Требования к отчету

Отчет должен содержать следующие разделы.

9.1. Тема и цель работы

Укажите название работы, ее цель и номер индивидуального варианта.

9.2. Условие задачи

Приведите полное условие и все исходные данные. Для табличных величин укажите единицы измерения.

9.3. Математическая модель

Представьте:

- переменные и параметры;
- целевую функцию;
- ограничения;
- начальные и терминальные условия;
- интерпретацию задачи как многоэтапного процесса.

9.4. Вывод соотношения Беллмана

Включите:

- словесное определение функции Беллмана;
- обоснование состава состояния;
- вывод рекуррентного соотношения;
- граничные условия;
- правило восстановления оптимальных решений.

9.5. Ручное решение

Приведите:

- хотя бы один подробно расписанный расчет для каждого типа этапа;
- полные расчетные таблицы для требуемого набора состояний;
- оптимальные управления;
- восстановление оптимального плана;
- проверку ограничений и целевой функции.

9.6. Программная реализация и тестирование

Включите:

- описание алгоритма;
- исходный код;
- результаты выполнения;
- сравнение с ручными вычислениями;
- результаты дополнительных тестов;
- оценку времени работы и объема памяти.

9.7. Выводы

В выводах укажите:

1. Какое оптимальное значение получено.
2. Какой план его обеспечивает.
3. Единственно ли найденное решение.
4. Совпали ли ручные и программные результаты.
5. Какие особенности постановки повлияли на состояние, рекуррентное соотношение и граничные условия.

10. Предлагаемые критерии оценивания

Критерий	Баллы
Корректность исходной математической постановки	15
Обоснованность выбора этапов, состояний и управлений	15
Вывод соотношения Беллмана и граничных условий	20
Правильность ручных расчетов	20
Восстановление и проверка оптимального плана	10
Программная реализация и тестирование	15
Качество оформления и выводов	5
Итого	100

В этой схеме оценивания программа не заменяет аналитическую часть: отсутствие вывода соотношения Беллмана или ручного решения означает, что соответствующий результат обучения не продемонстрирован.

11. Типичные ошибки

Ошибка	Способ устранения
Не определен смысл $V_k(s)$	Указать, какие этапы охватывает функция и что означает состояние
Состояние не содержит существенную информацию	Проверить, зависят ли дальнейшие решения от неучтенных событий прошлого
Вместо динамического программирования выбирается лучший текущий доход	Сравнивать непосредственный результат вместе с оптимальным результатом оставшихся этапов
Перепутаны состояние до этапа и состояние после него	Явно записать уравнение перехода
При обязательном полном распределении разрешен ненулевой конечный остаток	Исправить терминальное условие
Найдено оптимальное значение, но не найден план	Сохранять оптимальные управления и выполнить прямой проход

Ошибка	Способ устранения
При равных значениях потеряны альтернативные решения	Отразить равнозначные управления или явно указать правило выбора одного
Проверяется только итоговая сумма	Дополнительно проверить ограничения, переходы и промежуточные таблицы

12. Контрольные вопросы

1. Какие элементы необходимо определить для представления задачи как многоэтапного процесса?
2. Чем состояние системы отличается от управления?
3. Почему остаток ресурса является достаточным состоянием в рассмотренном примере?
4. Как изменится состояние, если доходность проектов зависит от ранее выбранных решений?
5. В чем состоит принцип оптимальности Беллмана?
6. Почему выбор максимального дохода текущего этапа не обязательно дает оптимальное решение всей задачи?
7. Как из содержательной постановки получить граничные условия?
8. Чем различаются обратный расчет функций Беллмана и прямое восстановление плана?
9. Как обнаружить наличие нескольких оптимальных решений?
10. Как изменится модель, если часть ресурса разрешено не использовать?
11. Как изменится алгоритм при ограничениях на максимальное финансирование каждого проекта?
12. Для чего при тестировании используется полный перебор?

⁂

1. https://economics.hse.ru/data/2019/12/14/1524220278/08.Динамическое программирование.pdf
2. https://web.mit.edu/15.053/www/AMP-Chapter-11.pdf
3. https://ece.iisc.ac.in/~aditya/E1245_Online_Prediction_Learning_F2014/ch2_DP_finite.pdf
4. https://web.mit.edu/6.7920/www/lectures/lec_3.pdf
5. https://ocw.mit.edu/courses/6-231-dynamic-programming-and-stochastic-control-fall-2015/pages/lecture-notes/
6. https://library.fa.ru/files/Kalihman1.pdf
7. https://dp.acmp.ru/index_files/present/dp_osn.pdf
8. https://studizba.com/show/1246769-1-bellman-r-prikladnye-zadachi.html
9. https://underactuated.mit.edu/dp.html
10. https://ru.scribd.com/document/665267690/Лежнёв-А-В-Динамическое-программирование-в-экономических-задачах

11. https://biblioclub.ru/index.php?page=book&id=447825
12. https://www.perplexity.ai/search/f318ea7f-0941-4ff8-8ec3-69e1cd232b15
13. https://www.mit.edu/~dimitrib/dpbook.html
14. https://ocw.mit.edu/courses/16-323-principles-of-optimal-control-spring-2008/resources/lec3/
15. https://togudv.ru/media/filer_public/2013/02/26/vlasenko_program.pdf
16. https://www.ektu.kz/files/DistanceEducation/Resource/777051/ОТУС тема 5 9 10 11 Крак.pdf